StepSpotter: OpenCV 5 decides what the agent does next

prepared by **Mikhail**

github.com/retailbox-automation/stepspotter-opencv · utgfmpcgxf.us-east-1.awsapprunner.com

> Numbers in double braces (`{{EVAL:…}}`) are filled from `docs/EVAL.md` and `eval/results/*.json` when the evaluation run is final. No number in this report comes from anywhere else.

1. Problem

StepSpotter walks one person through one small home repair, one step at a time, on a photo of their own thing, and opens the next step only after a photo proves the last one was done. Until this entry, the proof was one vision-model call on one photo (`src/stepspotter/verifier.py:61`). A single-photo model check has three blind spots that a better prompt does not fix:

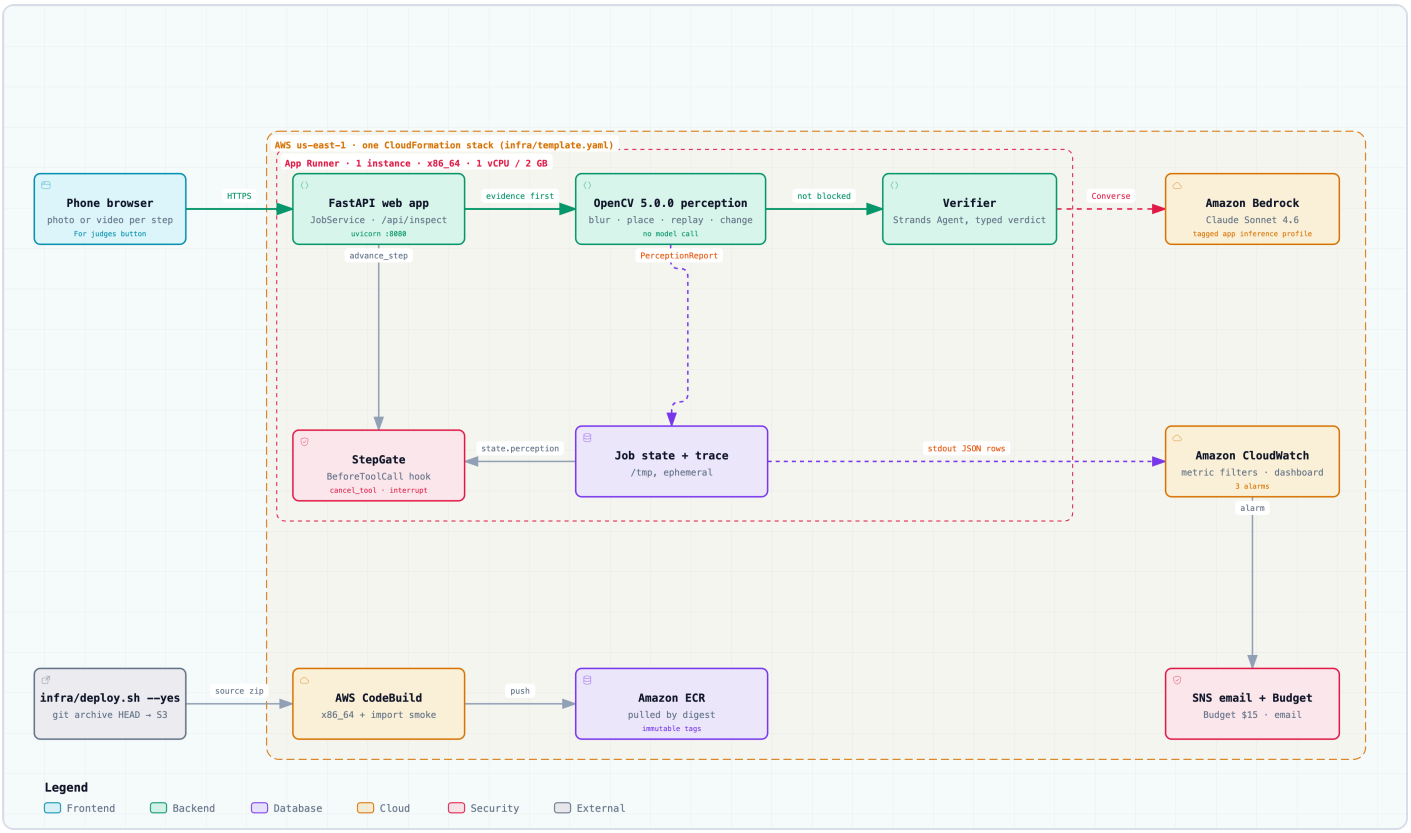
1. **A bad photo.** Blurred, dark, washed out, glare on the part, the part out of frame. The model either guesses or refuses without saying what to change, and every such attempt is a paid model call.
2. **The wrong photo.** One photo cannot tell the model that this is the same panel the job started on, or that this is the "before" photo sent again. A photo of a similar panel from somewhere else looks like valid proof.
3. **The wrong change.** The model does not compare before and after pixel by pixel: it cannot measure that the work zone changed and the do-not-touch zone did not.

OpenCV 5 now looks at every evidence photo first, without a model, and what it finds changes what the system does next: refuse with one instruction, send the photo to a one-photo or a two-photo model check, or stop the run for a person.

2. Users

People doing small, low-consequence jobs on their own home — low-voltage cabling, a network jack, a pressure-washer assembly — who learn best from their own photo marked up one step at a time. The first user is the author: the six evidence photos in this repository are of his own structured-wiring panel and wall jacks (`fixtures/perception/scenes.json`). StepSpotter's earlier build won the Everyday Agents Bronze award at the AWS "Agents for Humans" hackathon (see Disclosure); no other users are claimed. Jobs that need a licensed trade (mains wiring, gas, roofing, structural) are refused by the planner with an empty step list (`src/stepspotter/planner.py:26` , `:105`).

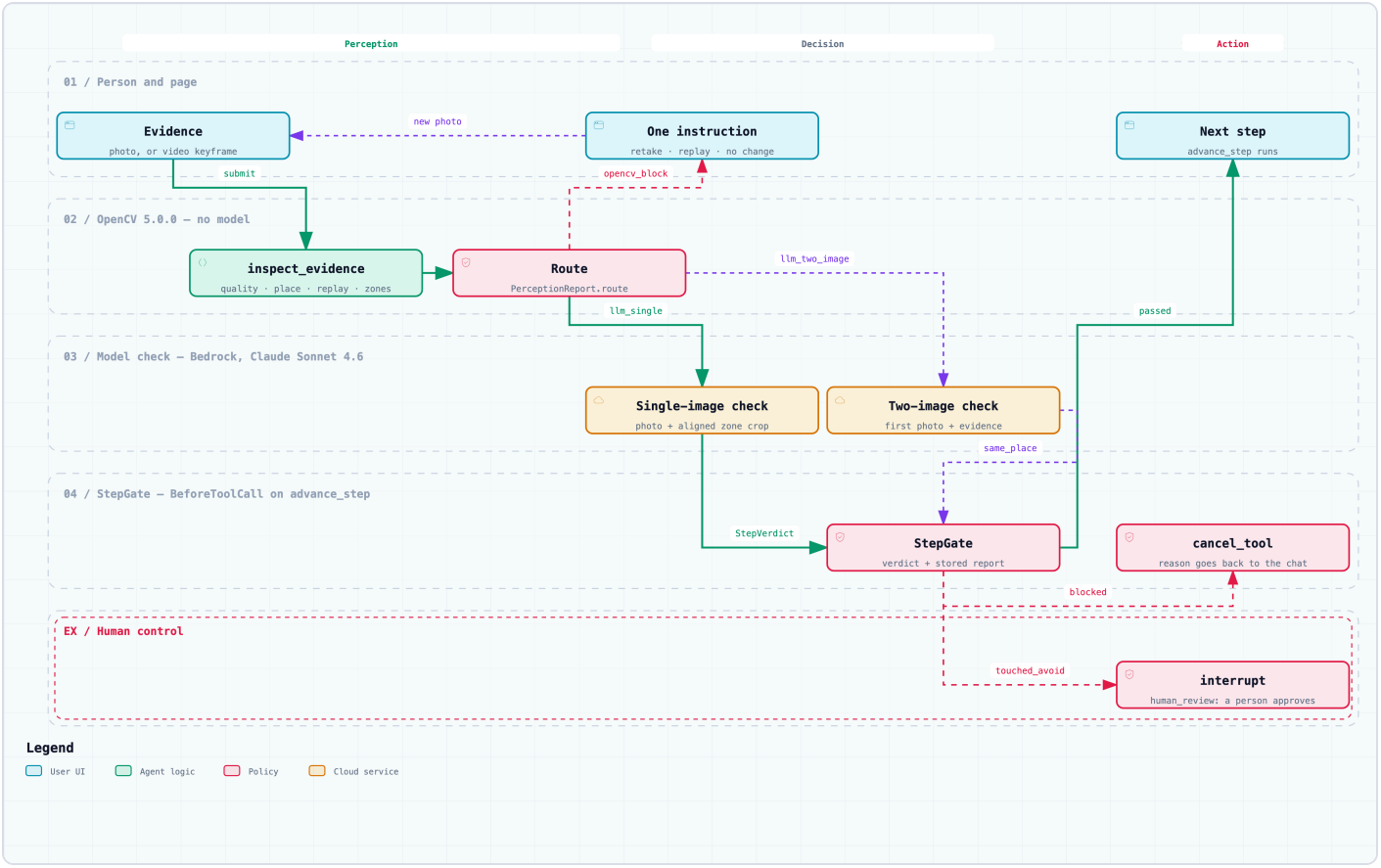
3. Architecture



Everything above runs in one container on AWS App Runner, deployed by one CloudFormation stack (`infra/template.yaml`). Every node in the diagram carries its source file and line in `docs/diagrams/architecture.html` (checked against commit `43c1520`).

- **FastAPI web app** (`src/stepspotter/web/app.py`) serves the phone page and the API. A photo goes to `JobService.verify` (`src/stepspotter/guide.py:396`).
- **OpenCV 5.0.0 perception** (`src/stepspotter/perception/`) runs first on every evidence photo and returns a `PerceptionReport` with a decision and a route.
- **Verifier** — a Strands `Agent` on Amazon Bedrock (Claude Sonnet 4.6) that returns a typed `StepVerdict` (`src/stepspotter/vision.py:65`, `src/stepspotter/verifier.py:61`). It is called only for photos OpenCV did not refuse.
- **StepGate** — a Strands `BeforeToolCall` hook on the `advance_step` tool (`src/stepspotter/gate.py:64`). The page's "next step" button calls `advance_step` through a Strands `HookRegistry` with this hook (`src/stepspotter/web/gated.py:23`), the same dispatch the agent loop uses.
- **Job state and trace** live on the instance's disk (`/tmp/stepspotter`); perception rows are also printed to stdout as JSON for CloudWatch (`src/stepspotter/store.py:113`).

The agent loop, as the competition's Agentic Vision award asks for it — perception, decision, action:



Two front doors use the same tools and the same gate. On the public page the person's buttons drive the loop and code routes each photo. In `stepspotter chat` (`src/stepspotter/cli.py:195`) a Strands Guide agent drives it itself and can call `inspect_photo` before submitting (`src/stepspotter/guide.py:654`); that agent is not exposed on the public URL.

4. OpenCV 5 implementation

Pinned: `opencv-python-headless==5.0.0.93`, `numpy==2.5.4` (`pyproject.toml`). The image build prints `cv2 5.0.0` from inside the built container (`infra/buildspec.yml:24`). Full description, every threshold and where it came from: `docs/PERCEPTION.md`.

check	OpenCV 5 calls	file
faces blurred before anything else reads the photo	`FaceDetectorYN` (YuNet, opencv_zoo), `GaussianBlur`	`perception/privacy.py:57`
sharpness, exposure, glare	`Laplacian`, `calcHist`, `threshold`, `connectedComponentsWithStats`	`perception/quality.py:65`, `:96`, `:80`, `:85`
same place as the first photo	`SIFT_create` (on CLAHE luminance), `BFMatcher.knnMatch`, `findHomography(USAC_MAGSAC)`, `perspectiveTransform`, `isContourConvex`, `intersectConvexConvex`	`perception/imgio.py:88`, `scene.py:55`, `:61`, `:78`, `:94`, `:101`

check	OpenCV 5 calls	file
the photo was already sent (replay)	homography near identity and aligned difference near zero	`perception/scene.py:247`
what changed in the work zone and in each do-not-touch zone	`warpPerspective`, `cvtColor(LAB)`, `createCLAHE`, `absdiff`, `morphologyEx`, `erode`, against the change outside all zones	`perception/change.py:40`, `:48`, `:54`, `:56`
a video instead of a photo	`VideoCapture`, best keyframe by sharpness × exposure × match	`perception/frames.py:30`, `:67`

The policy (`perception/inspect.py`, docstring, applied in order; the first rule that fires decides):

decision	route	next action
`retake` (blur, dark, washed out, glare, zone out of frame, blurrier than the first photo)	`opencv_block`	refused, one instruction to the person, **no model call**
`replay`	`opencv_block`	refused, no model call
`no_change` (the step had to change the work zone; it did not)	`opencv_block`	refused, no model call
`touched_avoid` (a do-not-touch zone changed)	`opencv_block`	the gate raises `interrupt("human_review")`
`place_unconfirmed` (OpenCV cannot line the photo up with the first photo)	`llm_two_image`	the model gets both photos and must answer `same_place`; the gate needs `same_place=true`
`ok`	`llm_single`	the model gets the photo and OpenCV's aligned crop of the work zone

Why `place_unconfirmed` is not a refusal. On our own photos, real re-shots of the same place from another angle kept 4–8 SIFT inliers after the homography fit, the same range as photos of different rooms (5–8); no inlier threshold separates them (`eval/results/opencv.md`, run on 2026-10-21 under the previous policy, which refused 3 of 5 such re-shots). So OpenCV confirms a place when it can (≥ 25 inliers, `perception/thresholds.py:115`) and otherwise hands the question to the model with both photos — a dearer check, never a refused person. The gate fails closed: a pass without `same_place=true` cannot close the step (`guide.py:440`, `gate.py:153`).

Where OpenCV output changes a later decision — the evidence the Agentic Vision award asks for, each with a test:

1. A blocking decision becomes a failed verdict with `source="opencv"` and the model is never called (`guide.py:421`; `tests/test_perception_gate.py`, `test_a_blurred_photo_is_refused_without_calling_the_model`).
2. The route chooses which model call is made: one photo plus crop, or two photos with a required `same_place` (`guide.py:434`; `test_a_photo_opencv_cannot_line_up_goes_to_the_two_photo_check`).
3. The gate cancels `advance_step` on a stored blocking report even beside a passing verdict (`gate.py:141`; `test_the_gate_refuses_on_a_blocking_report_even_beside_a_passing_verdict`).
4. A change in a do-not-touch zone turns the next `advance_step` into a request for a person's approval (`gate.py:91`; `test_a_change_in_the_do_not_touch_zone_stops_the_run_for_a_person`).
5. `inspect_photo` lets the agent ask once for a photo from the first angle before it submits (`guide.py:654`; `test_inspect_photo_offers_one_ask_for_the_first_angle_and_never_refuses`).

6. After three OpenCV blocks in a row on one step, the next `retake` or `no_change` goes to the model with OpenCV's findings attached, so a false positive cannot trap a person (`perception/inspect.py`, `apply_stuck_rule`; `tests/test_perception_failures.py`).

Every OpenCV decision is one trace row with `decision`, `route`, `llm_called` and `ms_total` (`guide.py:320`). On the page, "Everything it did" shows it as "OpenCV photo check (code, not the model)" with the match lines, the change heatmap and the time.

{{SCREENSHOT:trace — docs/screens/trace-opencv.png, a judge scenario's trace row on the live URL}}

---

5. AWS deployment

component	what it does here	where
AWS App Runner	one instance, 1 vCPU / 2 GB, x86_64, min = max = 1 (job state is on local disk)	`infra/template.yaml:440`, `:450`
Amazon Bedrock	Claude Sonnet 4.6 through a tagged application inference profile, so model spend is attributable to the stack	`infra/template.yaml:355`
IAM	instance role may invoke one model and put metrics into one namespace; no secrets in the environment	`infra/template.yaml:393`
CloudWatch	four metric filters on the JSON trace (`PerceptionDecisions` by decision, `LlmCallsSaved`, `LlmCallsMade`, `PerceptionLatencyMs`), three alarms (5xx, request flood, perception p95), one dashboard	`infra/template.yaml:502` – `654`
SNS + AWS Budget	alarm email; $15/month Budget created before the service	`infra/template.yaml:160`, `:177`
CodeBuild + ECR + S3	`git archive HEAD` → S3 → x86_64 build with an architecture check and an import smoke → ECR (immutable tags, scan on push) → App Runner by digest	`infra/deploy.sh:156`, `infra/buildspec.yml`, `infra/template.yaml:212`, `:322`

One command deploys (`infra/deploy.sh --yes`) and one removes (`infra/destroy.sh --yes`); full procedure, measured durations and cost in `docs/DEPLOY.md`, operations and the kill switch in `docs/OPERATIONS.md`. Spend is bounded in code: per-address caps (6 jobs, 30 photo checks an hour) and global daily caps (6 jobs, 30 photo checks a day) bound Bedrock at ≤ 78 calls a day (`src/stepspotter/web/limits.py`, `docs/OPERATIONS.md`). A photo OpenCV refuses costs no model call. Model price used: $3 / $15 per million input / output tokens for Claude Sonnet 4.6 in us-east-1 (AWS Price List API, effective 2026-09-01, read 2026-10-21); at the 3,142 / 657 average tokens per call measured on the first deployment that is about $0.019 per model call (`docs/DEPLOY.md`).

Deployed build: {{DEPLOY:commit and image digest of the final redeploy}}. OpenCV latency on the App Runner instance: {{EVAL:cloud_latency — p50/p95 of ms_total on AWS, from a recorded run}}.

---

6. Evaluation

Full method, data provenance and every failure: `docs/EVAL.md`. Summary:

Data. {{EVAL:set_summary — cases by source (own photos, own video frames, public photos with licence, synthetic damage) and by class; calibration / test split by scene}}

Configurations. A = model only, one photo · A′ = model only, first photo and evidence · B = OpenCV only · C = OpenCV then model (the shipped gate).

{{EVAL:table_ABC — per class: caught / missed on bad evidence, false blocks on good evidence, for A, A′, B, C; test half only}}

	A	A′	B	C
bad evidence caught	{{EVAL:A_caught}}	{{EVAL:A2_caught}}	{{EVAL:B_caught}}	{{EVAL:C_caught}}
good evidence refused	{{EVAL:A_false}}	{{EVAL:A2_false}}	{{EVAL:B_false}}	{{EVAL:C_false}}
model calls	{{EVAL:A_calls}}	{{EVAL:A2_calls}}	0	{{EVAL:C_calls}}
latency p50 / p95	{{EVAL:A_ms}}	{{EVAL:A2_ms}}	{{EVAL:B_ms}}	{{EVAL:C_ms}}
$ per 100 checks	{{EVAL:A_cost}}	{{EVAL:A2_cost}}	0	{{EVAL:C_cost}}

Where C decided differently from A and A′. {{EVAL:c_differs — each case with the image pair, OpenCV's numbers and both decisions}}

Real re-shots of the same place. {{EVAL:false_blocks_real_reshots}}

How to read the synthetic classes. Synthetic damage is of a known kind and size and the checks were built for it, so a high catch rate there shows the checks fire on what they target, not field accuracy. The informative numbers are the false refusals, the real pairs and the comparison with A′.

---

7. Limitations and failure cases

- **Real re-shots do not line up.** A photo of the same place from another angle or after cables moved often keeps too few SIFT inliers to confirm the place. Since `793b987` it costs a two-photo model call instead of a refusal; before that, OpenCV refused 3 of 5 such pairs in our set (`eval/results/opencv.md`). OpenCV alone can no longer reject a photo of another room; only the model can, through `same_place`.
- **Parallax.** On the real 01↔02 pair, alignment left 37 % of the frame "changed"; such maps are marked `unreliable` and decide nothing, so `no_change` and `touched_avoid` fire only for photos taken from near the first viewpoint (`docs/PERCEPTION.md`).
- **Zones come from a model.** Work and do-not-touch zones are the Marker model's boxes, which drift and run oversized (`src/stepspotter/marker.py`); a wrong zone makes a right change look wrong.
- **Thresholds were set on six photos of one house** and are not a blind test; the scene split shows whether results differ by scene, not field accuracy.
- **Faces:** checked on one public-domain portrait (`tests/test_privacy_faces.py`); no detection rate is claimed.
- **One instance, local disk.** Jobs live on the App Runner instance and are lost on every redeploy, pause or instance replacement (`docs/OPERATIONS.md`).
- **Judging capacity.** The global daily caps (6 jobs, 30 photo checks a day, shared by everyone) protect the budget and can run out on a busy judging day; the offline test suite and the eval harness need no AWS account.
- {{EVAL:opencv_failures — OpenCV misses and false refusals from the final run, with pictures and reasons}}

8. Responsible use

- **Scope.** Low-voltage, low-consequence jobs only. The planner refuses mains wiring, gas, roofing and structural work with an empty step list (`planner.py:26`). That refusal is a model judgement, not a certified hazard classifier.
- **The person stays in control of risk.** A change in a do-not-touch zone stops the run for a person's approval (`interrupt`, not a cancel), and a hazard seen by the model stops the run too (`gate.py:76`). A refusal always comes with the reason and one instruction; the stuck rule keeps a false positive from trapping anyone.
- **Fail closed.** A photo whose place OpenCV cannot confirm closes a step only if the model says it is the same place (`guide.py:440`, `gate.py:153`).
- **Privacy.** Faces are blurred in the stored file before any model sees it; each report says whether that check ran (`faces_checked`). Photos live only on the instance's ephemeral disk, are not copied to S3 or a database, and are gone on every redeploy. Model-invocation logging is not turned on. CloudWatch gets perception metrics and decisions, no image data; rate-limit rows carry a salted hash of the caller address (`docs/OPERATIONS.md`, `src/stepspotter/store.py`).
- **Spend and abuse.** Per-address and global daily caps, a kill switch (`STEPSPOTTER_PAUSED=1`), a Budget alert and a request-flood alarm.
- **Least privilege.** The instance role can call one model and write one metric namespace; the build role can push to one repository (`infra/template.yaml:271`, `:393`).
- **Data rights.** Our own photos are published with the owner's permission; the face test photo is public domain (`tests/fixtures/ATTRIBUTION.md`); public photos in the evaluation carry their licence and attribution (`eval/cases/public/ATTRIBUTION.md`). The repository's MIT licence does not cover third-party photos.
- **Honest results.** Failures are published next to successes, and every number traces to a committed results file.

9. Disclosure

StepSpotter was built on 9–14 September 2026 for the AWS "Agents for Humans" hackathon on Devpost, where it won the Everyday Agents Bronze award (announced 14 October 2026). It was also submitted to the AI Builders hackathon, whose judging has not finished. The original code is public at https://github.com/retailbox-automation/stepspotter, commit `80979af` (pushed 2026-09-14). It is imported here as the first commit, `295e3f3`, with the text copied from manufacturers' manuals removed (the bundled manual cache keeps only links and page numbers).

Everything OpenCV 5 — the perception package, its integration into the gate and the verifier, the face blurring, the evaluation harness, the AWS stack, the diagrams and this report — was built on 21–24 October 2026, inside this competition's window: commits `9cc6ca1` through **{{FINAL:head commit}}** (`git log --reverse 295e3f3..HEAD`).

We read the Official Rules for a clause on previously submitted projects and found none. The nearest is the timeline: «From August 26 through October 26, teams will build, evaluate, document, and submit their final projects.» The rules also say «Disclosed agents and automated programs that are integral to the submitted project are allowed and encouraged».

10. How AI was used

- **In the product:** Claude Sonnet 4.6 on Amazon Bedrock plans the steps, draws the zones and checks the photos OpenCV lets through (one photo, or two photos with `same_place`); a tool-less agent answers questions typed into the chat. OpenCV runs before it on every evidence photo.

- **In development:** the code, tests, infrastructure and these documents were written with AI coding assistants, directed and reviewed by the author. The author chose the product and its rules, took the photos, decided what to publish and approved every spend and every outward step.

Details: `docs/AI-USE.md`.

Reproduce

```
git clone https://github.com/retailbox-automation/stepspotter-opencv && cd stepspotter-opencv
python3.12 -m venv .venv && .venv/bin/pip install -e ".[dev]"
tools/fetch_models.sh                                  # YuNet face detector, SHA256-checked
.venv/bin/python -m pytest -q -m "not integration"     # offline, no AWS account
.venv/bin/python eval/build_set.py && .venv/bin/python eval/run.py --config opencv
infra/deploy.sh --yes                                  # needs an AWS account; see docs/DEPLOY.md
```